UVa SEGOVIA

inf5G INFORMÁTICA ESCUELA INGENIERÍA SEGOVIA

CAMPUS PÚBLICO MARÍA ZAMBRANO SEGOVIA

Universidad de Valladolid

Lab 3.
Manejo de eventos

[DISEÑO DE INTERFACES DE USUARIO]

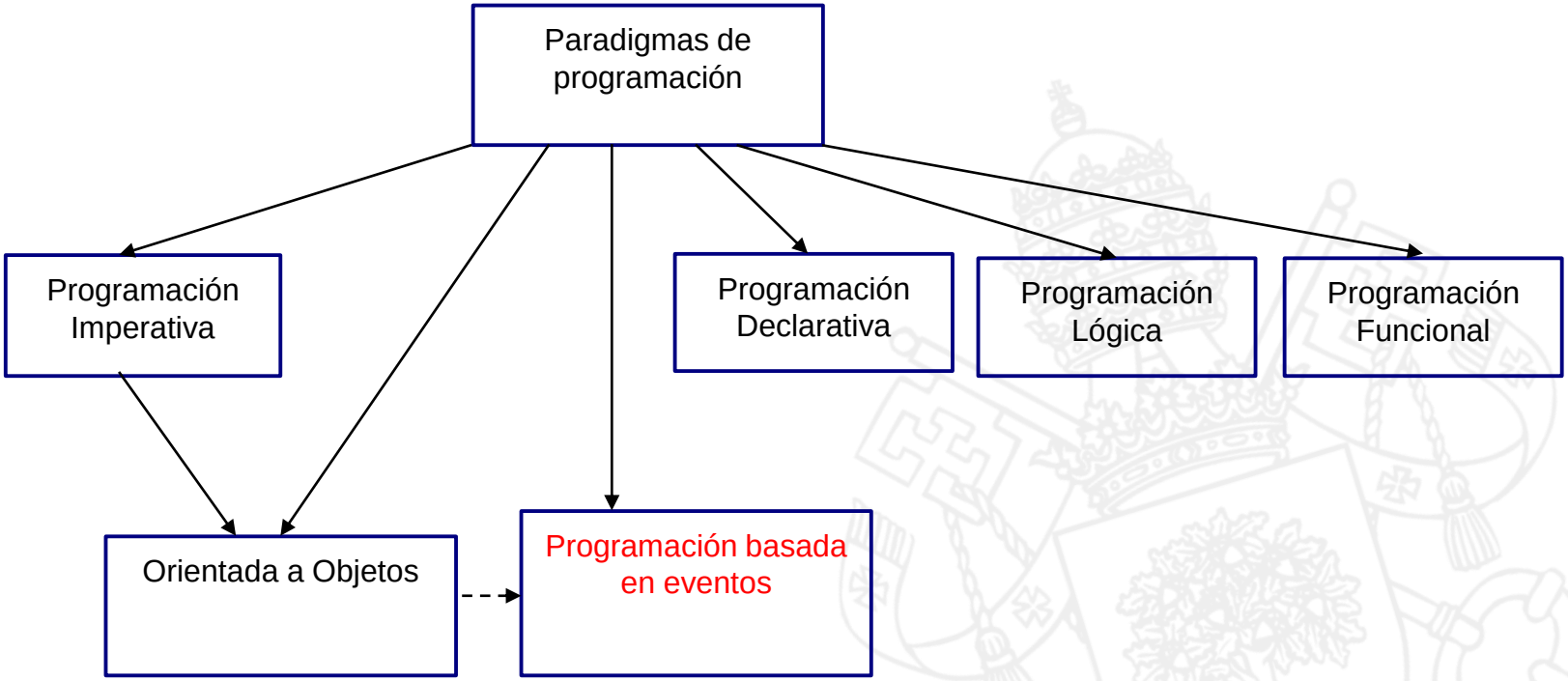
Víctor Martínez Cagigal

Revisión: Nicolás Sacristán Machín

Manejo de eventos

- Cada vez que un usuario pulsa una tecla o un botón del ratón o activa una componente de la GUI se genera un **evento**.
- Cualquier objeto gráfico puede ser fuente de un evento.
- Esta forma de trabajar define un nuevo paradigma de programación.

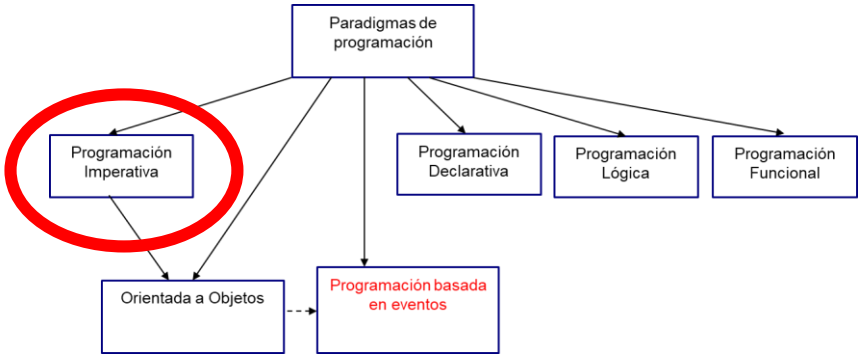
Paradigmas de programación



Paradigmas de programación

Programación imperativa

- Programación imperativa
  - Paradigma más antiguo
  - Establecemos una secuencia claramente definida de instrucciones
  - Describimos lo que queremos lograr en términos de "cómo" hacerlo
  - E.g.: PHP, Python, JavaScript



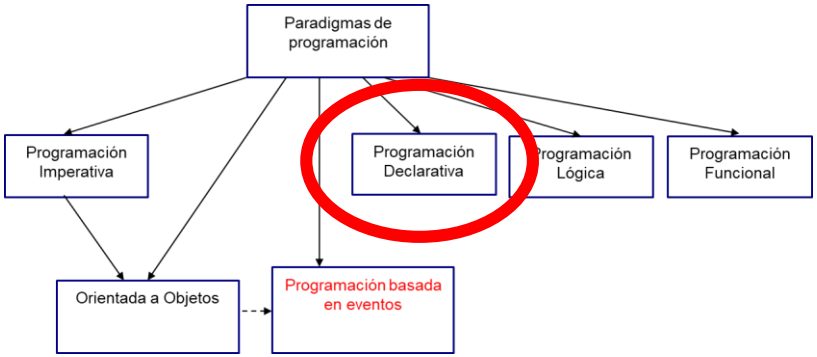

```javascript
var people = [
  {
    name: 'Aang',
    age: 12,
  },
  {
    name: 'Sokka',
    age: 15,
  },
  {
    name: 'Toph',
    age: 12,
  },
  {
    name: 'Iroh',
    age: 55,
  }
];
var kid_names = [];
for (let person of people) {
  if (person.age <= 12) {
    kid_names.push(person.name);
  }
}
console.log(kid_names);
```

Paradigmas de programación

Programación declarativa

- Programación declarativa
  - Especificamos lo que queremos obtener en lugar del "cómo"
  - Es un nivel de abstracción más alto que el paradigma imperativo
  - E.g., SQL

```
SELECT name FROM people AS kids WHERE age <= 12;
```

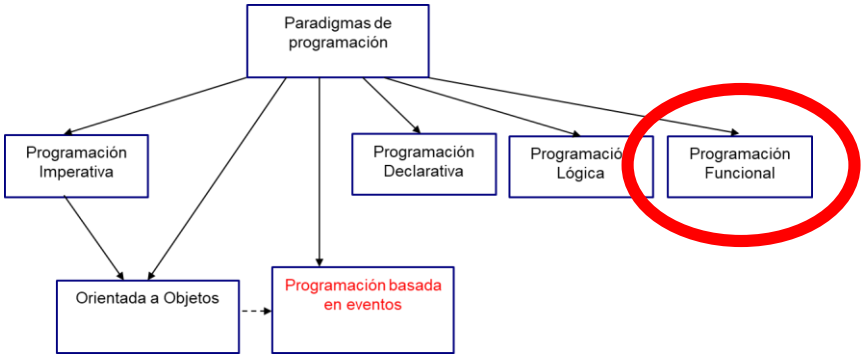


Paradigmas de programación

Programación funcional

- Programación funcional
  - Permite escribir código declarativo en lenguajes imperativos
  - Normalmente mediante funciones de orden superior y callbacks (e.g., filter, map)

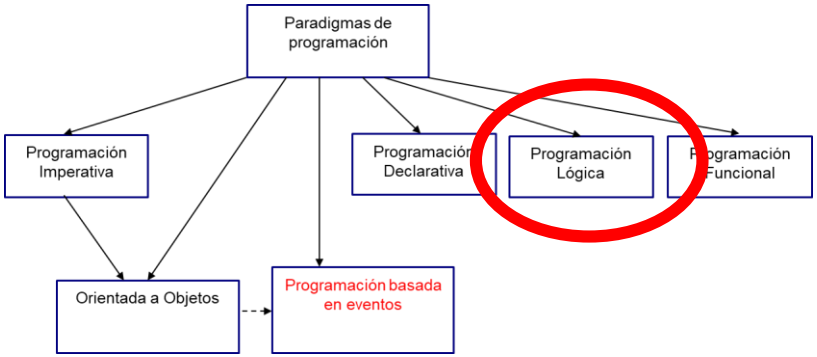
```
function isAKid(person) {
  return person.age <= 12;
}
function getName(person) {
  return person.name
}
var kid_names = people
    .filter(function isAKid(person) {
        return person.age <= 12;
    })
    .map(function getName(person) {
        return person.name
    });
```



Paradigmas de programación

Programación lógica

- Programación lógica
  - Subtipo de la programación declarativa
  - Se emplean operadores lógicos como "and", "not", "or"
  - Algoritmos = lógica + control
  - Axiomas → lógica
  - Problema → control
  - E.g., Prolog, Lisp, Erlang



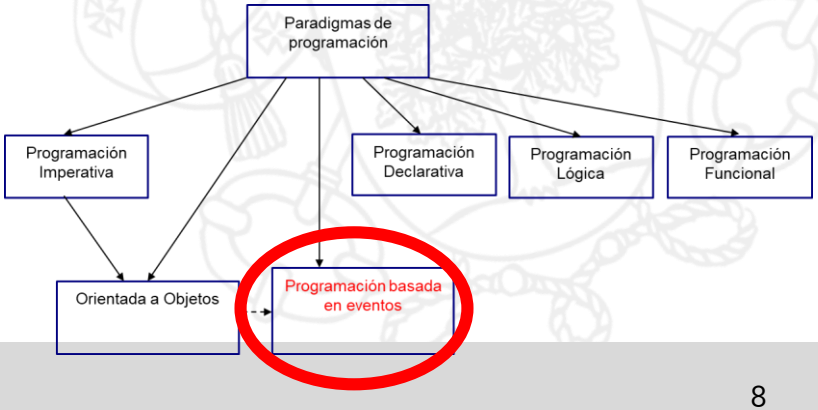

```
es_hijo(Juanito, Juan)

suma(10,5,15)

?es_hijo(x,Juan).
```

¿Existe un X que sea hijo de Juan?

Paradigmas de programación

Programación basada en eventos

- Programación basada en eventos
  - Los programas de consola típicos siguen un flujo secuencial en el que típicamente se tienen ciclos: entrada → procesamiento → salida
  - El diseño de una GUI debe tener en cuenta la variedad de posibles interacciones con el usuario.
  - En lugar de un único flujo de entrada de datos por consola, las GUIs permiten muchas más acciones de usuario
    - ✓ e.g. presionar botones gráficos, escribir texto en un campo de texto, o mover un scrollbar.
  - ¿Cómo podemos estar atento a tantas cosas al mismo tiempo?
    - ✓ Respuesta: Manejo de eventos.



Paradigmas de programación

Programación basada en eventos

- Programación basada en eventos
  - La ejecución se determina por los sucesos que ocurran en el sistema → el usuario genera eventos
  - Al comenzar, se inicializan las acciones y el programa se bloquea hasta que se produzca un evento

```
JLabel label = new JLabel("hello");
Jbutton btn = new Jbutton();
btn.addActionListener(new ActionListener() {
  public void actionPerformed(ActionEvent e) {
    label.setText("clicked");
  }
});
```

Objetos implicados en un evento

- El objeto **fuente** → Es el objeto que lanza el evento. Dependiendo del tipo de objeto así será el evento.

- El objeto **escuchador** (*Listener*) → Se trata del objeto que recibe el evento producido y ejecuta el código correspondiente al interfaz relacionada con el tipo del evento. En este código se especifican las acciones a realizar.

- El objeto **evento** → Es el objeto enviado desde el objeto fuente al objeto *Listener*. Según el tipo de evento se ejecutará uno o otro método del *Listener*.

Implementación de un manejador de eventos

- Cada manejador de eventos consta de tres piezas de código:
  1. Declaración de la clase del manejador de eventos que deberá implementar el interfaz del *Listener* o extender una clase que implemente dicho interfaz.

```
public class MiClase extends JFrame implements ActionListener{…
```

  2. Registro del objeto asociado al interfaz del Listener (objeto Listener) vinculado al objeto fuente.

Objeto Fuente

Objeto Listener

```
Componente.addActionListener(instanciadeMiClase);
```

  3. Implementación de los métodos definidos por el interfaz del "Listener". En estos métodos se definen las acciones a ejecutar en función del objeto evento capturado por el objeto Listener.

```
public void actionPerformed(ActionEvent e) {
    ...
//código que define la acción a ejecutar en función del evento
```

Objeto Evento

Ejemplo 1. Un ejemplo sencillo

```
import javax.swing.SwingUtilities;
import java.awt.event.*;
import java.awt.*;
import javax.swing.*;

public class Ej1_ClickMe extends JFrame implements ActionListener {
    JButton b = new JButton("Click me!");

    public Ej1_ClickMe() {
        setTitle("Ejemplo 1");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        b.addActionListener(this);
        add(b);

        pack();
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        b.setBackground(Color.CYAN);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(Ej1_ClickMe::new);
    }
}
```

Objeto Fuente

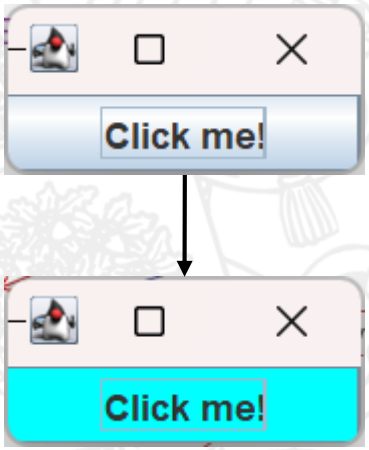
Interfaz implementada

Objeto Listener

Registro del Listener

Objeto Evento

Definición de la acción
una vez se captura el evento



Ejemplo 2. Otro ejemplo de evento

```
import javax.swing.SwingUtilities;
import java.awt.event.*;
import javax.swing.*;

public class Ej2_ContadorClicks extends JFrame {
    private static String labelPrefijo = "Número de clicks sobre el botón: ";
    private int numClicks = 0;
    private JLabel label = new JLabel(labelPrefijo + "0    ");

    public Ej2_ContadorClicks() {
        setTitle("Ejemplo 2");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JButton button = new JButton("Cuenta");
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                label.setText(labelPrefijo + ++numClicks);
            }
        });
        JPanel panel = new JPanel();
        panel.add(button);
        panel.add(label);
        add(panel);

        pack();
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(Ej2_ContadorClicks::new);
    }
}
```

Registro del Listener

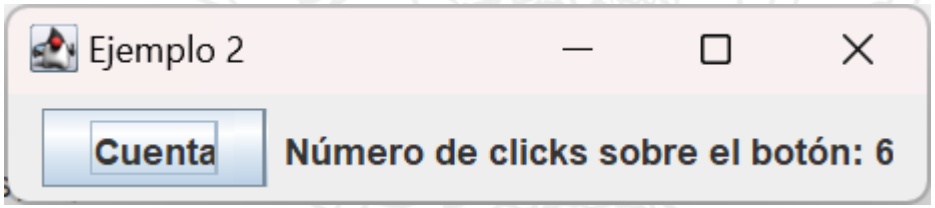
Objeto Fuente

Objeto Listener

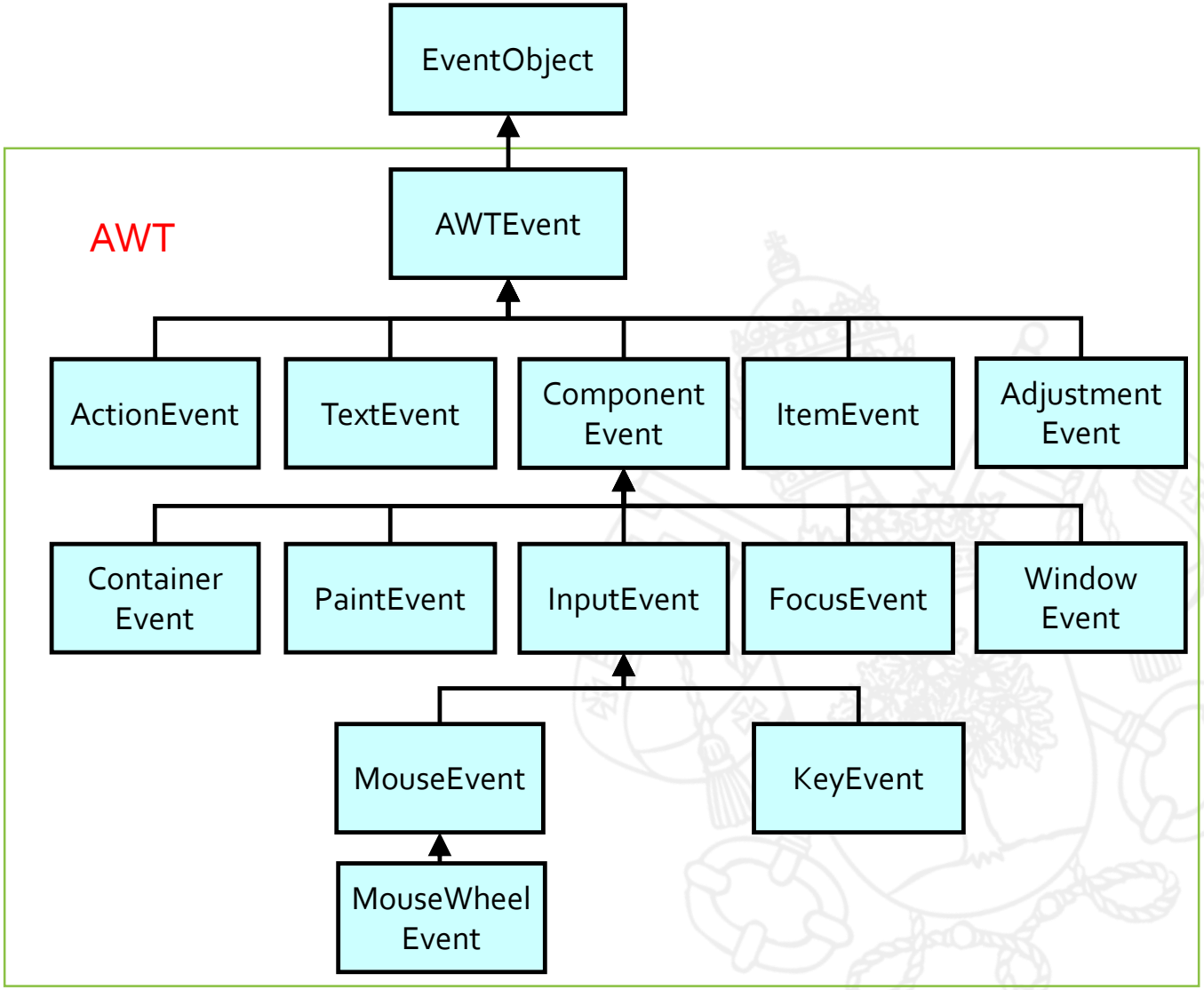
Implementación dinámica del Interfaz del Listener

Objeto Evento

Definición de la acción una vez se “escucha” el evento



Jerarquía del paquete java.awt.event



EventObject: métodos

método	definición
Object **getSource()**	Devuelve el objeto que lanzó el evento.
String **toString()**	Redefinición del método de la clase Object que devuelve un string con la información sobre el evento.

AWTEvent: métodos

método	definición
void **setSource(Object o)**	Redirige el evento al objeto *o*, haciendo que sea éste el que lance de nuevo el evento.
protected void **consume()**	Consume el evento. Esto es, hace que el evento deje de ser enviado. No todos los eventos lo permiten.
protected boolean **isConsume()**	Indica si el evento ya ha sido consumido.

InputEvent: métodos

método	definición
int **getModifiers()**	Devuelve un entero que representa una máscara de bits donde los bits representan con valor 1, teclas de control y botones de ratón pulsado. Por esta razón se deben comparar con máscaras.
int **getModifiersExt()**	Versión moderna de la anterior (a partir de la versión 1.4)
void **consume()**	Deriva de AWTEvent pero en este caso el método es público.
boolean **isConsume()**	Deriva de AWTEvent pero en este caso el método es público.
long **getWhen()**	Devuelve el momento exacto en el que ocurrió el evento. Lo devuelve en formato long (que representa los milisegundos acaecidos). Habría que convertirlo a objeto Date (mediante el constructor new Date(long)) para así poder obtener hora, minutos y segundos (y también la fecha)
boolean **isAltDown()**	Devuelve true si la tecla **Alt** estaba pulsada cuando se produjo el evento.
boolean **isAltGraphDown()**	Devuelve true si la tecla **AltGraph** estaba pulsada cuando se produjo el evento.
boolean **isControlDown()**	Devuelve true si la tecla **Control** estaba pulsada cuando se produjo el evento.
boolean **isShiftDown()**	Devuelve true si la tecla **Shift** estaba pulsada cuando se produjo el evento.

InputEvent: máscaras

- Constantes de máscaras predefinidas en la clase InputEvent:
  - **SHIFT_MASK, ALT_MASK, ALT_GRAPH_MASK, BUTTON1_MASK, BUTTON2_MASK, BUTTON3_MASK**
- Actualmente se utiliza otro formato que incluye la palabra DOWN.
  - **ALT_DOWN_MASK,**
  - **ALT_GRAPH_DOWN_MASK**
  - **.....**
  - **Estas últimas aparecen en la versión 1.4. y se utilizan con el método getModifiersExt.**
- Ejemplo:

```
public void mouseReleased(MouseEvent e){
    int valor=e.getModifiers();
    if((valor & InputEvent.SHIFT_MASK)!=0){
    //La tecla Mayúsculas (Shift) se pulsó con el ratón
    ......
```

Tipos de eventos (AWT)

- **ComponentEvent**. java.awt.event.ComponentEvent. Ocurre cuando un componente se mueve, cambia su tamaño o su visibilidad.
- **FocusEvent**. java.awt.event.FocusEvent. Ocurre cuando cambia el foco de atención de una componente a otra.
- **KeyEvent**. java.awt.event.KeyEvent. Ocurre cuando se pulsa una tecla (y el componente posee el foco de atención). Es un evento del tipo InputEvent.
- **MouseEvent**. java.awt.event.MouseEvent. Ocurre cuando se pulsa el ratón sobre un componente. Es un evento del tipo InputEvent.
- **MouseMotion**. Sirve para capturar objetos MouseEvent igual que en el caso anterior, solo que en este caso permite capturar eventos de arrastre y movimiento del ratón.

Tipos de eventos (AWT)

- **MouseWheelEvent**. java.awt.event.MouseWheelEvent. Sirve para capturar eventos de movimiento de rueda de ratón (en los ratones que posean esta característica). Implementado en la versión 1.4.
- **WindowEvent**. java.awt.event.WindowEvent. Captura eventos de ventana.
- **ContainerEvent**. java.awt.event.ContainerEvent. Ocurre cuando se añaden o quitan controles a un contenedor del tipo que sea.
- **ActionEvent**. java.awt.event.ActionEvent. Se ejecuta una orden de acción.
- **AdjustmentEvent**. java.awt.event.AdjustmentEvent. Ocurre cuando se mueve el scroll de la pantalla.
- **ItemEvent**. java.awt.event.ItemEvent. Ocurre cuando se cambia el estado de un control de tipo ítem.

ActionEvent

ActionEvent	java.awt.ActionEvent
Objetos que lo lanzan	JButton, JCheckBoxMenuItem, JComboBox, JFileChooser, JRadioButtonItem, JTextField, JToggleButton
Método lanzador de eventos	void **addActionListener(ActionListener objetoListener)**
Interfaz que lo implementa	ActionListener.
Métodos de control	void **actionPerformed(ActionEvent objetoEvento)**
Otros métodos interesantes	String **getActionCommand()** Devuelve una cadena que identifica la acción

WindowEvent

WindowEvent	java.awt.WindowEvent
Objetos que lo lanzan	JDialog, JWindow, JFrame
Método lanzador de eventos	void **addWindowListener(WindowListener objetoListener)**
Interfaz que lo implementa	WindowListener.
Métodos de control	void **windowOpened(WindowEvent** e**)** La ventana se está abriendo void **windowClosing(WindowEvent** e**)** La ventana se está cerrando void **windowClosed(WindowEvent** e**)** La ventana se cerró void **windowIconified(WindowEvent** e**)** Se minimizó la ventana void **windowDeiconified(WindowEvent** e**)** Se restauró la ventana void **windowActivated(WindowEvent** e**)** Se activó la ventana

MouseEvent

MouseEvent	java.awt.MouseEvent
Objetos que lo lanzan	Cualquier componente
Método lanzador de eventos	void **addMouseListener(MouseListener objetoListener)**
Interfaz que lo implementa	MouseListener.
Métodos de control	void **mouseClicked (MouseEvent objetoEvento)** Se hizo clic con un botón del ratón en el componente void **mousePressed(MouseEvent objetoEvento)** Se pulsó un botón del ratón void **mouseReleased(MouseEvent objetoEvento)** Se levantó un botón del ratón void **mouseEntered(MouseEvent objetoEvento)** El ratón está sobre el componente. void **mouseExited(MouseEvent objetoEvento)** El ratón salió fuera de el componente
Otros métodos interesantes	**int getX()** Devuelve la posición X de pantalla del ratón **int getY()** Devuelve la posición Y de pantalla del ratón **Point getPoint()** Devuelve la posición del cursor de ratón en forma de objeto Point. **int getClickCount()** Devuelve el número de clics seguidos realizados por el ratón **int getButton()** Indica qué botón del ratón se pulsó. Se pueden usar estas constantes: MouseEvent.BUTTON1, MouseEvent.BUTTON2 y MouseEvent.BUTTON3, MouseEvent.NOBUTTON. A partir de la versión 1.4 de Java.

KeyEvent

KeyEvent	java.awt.KeyEvent
Objetos que lo lanzan	Cualquier componente
Método lanzador de eventos	void **addKeyListener(MouseListener objetoListener)**
Interfaz que lo implementa	KeyListener.
Métodos de control	void **keyTyped(KeyEvent objetoEvento)** Ocurre cuando se pulsa una tecla que escribe (una tecla que escriba un símbolo Unicode, la tecla F1 por ejemplo no valdría) void **keyPressed(KeyEvent objetoEvento)** Ocurre cuando se pulsa cualquier tecla void **keyReleased(KeyEvent objetoEvento)** Ocurre cuando se levanta cualquier tecla
Otros métodos interesantes	char **getKeyChar**() Devuelve el código Unicode de la tecla pulsada int **getKeyCode**() Devuelve el código entero de la tecla pulsada, útil para capturar teclas de control. Se pueden utilizar constantes del objeto *KeyEvent* para comparar. Esas constantes empiezan con el signo VK_ seguida del nombre de la tecla en mayúsculas, por ejemplo KeyEvent.VK_F1 **representa** la tecla F1. char **getKeyText**(**int** código) Devuelve la traducción del código de la tecla a formato de texto boolean **isActionKey()** Devuelve true si el evento lo provocó una tecla de acción. Una tecla de acción es una tecla que no escribe y que no modifica otras teclas (como *Shift* o *Alt*)

FocusEvent

FocusEvent	java.awt.FocusEvent
Objetos que lo lanzan	Cualquier componente.
Método lanzador de eventos	void **addFocusListener(FocusListener objetoListener)**
Interfaz que lo implementa	FocusListener.
Métodos de control	void **focusGained(FocusEvent objetoEvento)** Ocurre cuando se obtiene el foco de atención void **focusLost(FocusEvent objetoEvento)** Ocurre cuando se pierde el foco de atención
Otros métodos interesantes	Component **getOppositeComponent()** Devuelve el otro componente involucrado en el foco. Si el objeto actual gana el foco de atención, el objeto involucrado es el que lo perdió. boolean **isTemporary()** Con valor true indica que el cambio de foco de atención es temporal.

AdjustmentEvent

AdjustmentEvent	java.awt.AdjustmentEvent
Objetos que lo lanzan	ScrollBar
Método lanzador de eventos	void **addAdjustmentListener(AdjustmentListener objetoListener)**
Interfaz que lo implementa	AdjustmentListener.
Métodos de control	void **adjustmentValueChanged(AdjustmentEvent objetoEvento)** Ocurre cuando se produce un cambio en el valor del ScrollBar
Otros métodos interesantes	int **getValue()** Devuelve el valor actual del ScrollBar

Tipos de eventos (Swing)

- **AncestorEvent**. javax.swing.event.AncestorEvent. Se genera cuando un ancestro de una componente ha sido añadido, movido o removido.
- **CaretEvent**. javax.swing.event.CaretEvent. Se genera cuando la posición del signo de intercalación (cursor) en una componente de texto cambia.
- **HyperLinkEvent**. javax.swing.event.HyperlinkEvent. Ocurre cuando le sucede algo a un texto de enlace (hipervínculo), un clic, posar el ratón,…
- **ListSelectionEvent**. javax.swing.event.ListSelectionEvent. Cambio en la selección de un control de lista.
- **MenuEvent**. javax.swing.event.MenuEvent. Cambio en la selección de un menú.

Tipos de eventos (Swing)

- **PopupMenuEvent**. javax.swing.event.PopupMenuEvent. Cambio en la selección de un menú de tipo popup.
- **MenuKeyEvent**. javax.swing.event.MenuKeyEvent. Ocurre cuando se pulsaron teclas hacia el menú.
- **MenuDragMouseEvent**. javax.swing.event.MenuDragMouseEvent. Ocurre cuando se arrastró el ratón sobre un elemento de menú.
- **ChangeEvent**. javax.swing.event.ChangeEvent. Cambio en la selección de un control de lista.
- **InternalFrameEvent**. java.awt.event.InternalFrameEvent. Ocurre cuando hay cambios en objetos de ventana interna.

ChangeEvent

ChangeEvent	javax.swing.ChangeEvent
Objetos que lo lanzan	Cualquier componente.
Método lanzador de eventos	void **addChangeListener(ChangeListener objetoListener)**
Interfaz que lo implementa	ChangeListener.
Métodos de control	void **stateChanged(ChangeEvent objetoEvento)** Ocurre cuando cambia el estado del objeto
Otros métodos interesantes	Component **getSource()** Devuelve la componente involucrada en el evento.

CaretEvent

CaretEvent	javax.swing.CaretEvent
Objetos que lo lanzan	Cualquier componente de Texto.
Método lanzador de eventos	void **addCaretListener(CaretListener objetoListener)**
Interfaz que lo implementa	CaretListener.
Métodos de control	Void **caretUpdate(CaretEvent objetoEvento)** Ocurre cuando cambia el estado del cursor
Otros métodos interesantes	int **getDot()** Devuelve la posición actual del cursor int **getMark()** Devuelve la posición inicial actual de la selección

Adaptadores

- Para facilitar la gestión de eventos en ciertos casos, Java dispone de clases abstractas llamadas clases adaptadoras.
- De esta forma no es necesario implementar todos lo métodos definidos en los interfaces.
- Todas poseen la palabra adapter en el nombre de clase:
  - ComponentAdapter
  - ContainerAdapter
  - FocusAdapter
  - InternalFrameAdapter
  - KeyAdapter
  - MouseAdapter
  - MouseMotionAdapter
  - PrintJobAdapter
  - WindowAdapter

MouseAdapter

- Definición:

```
public abstract class MouseAdapter implements MouseListener{
  public void mouseClicked(MouseEvent e) {}
  public void mousePressed(MouseEvent e) {}
  public void mouseReleased(MouseEvent e) {}
  public void mouseEntered(MouseEvent e) {}
  public void mouseExited(MouseEvent e) {}
}
```

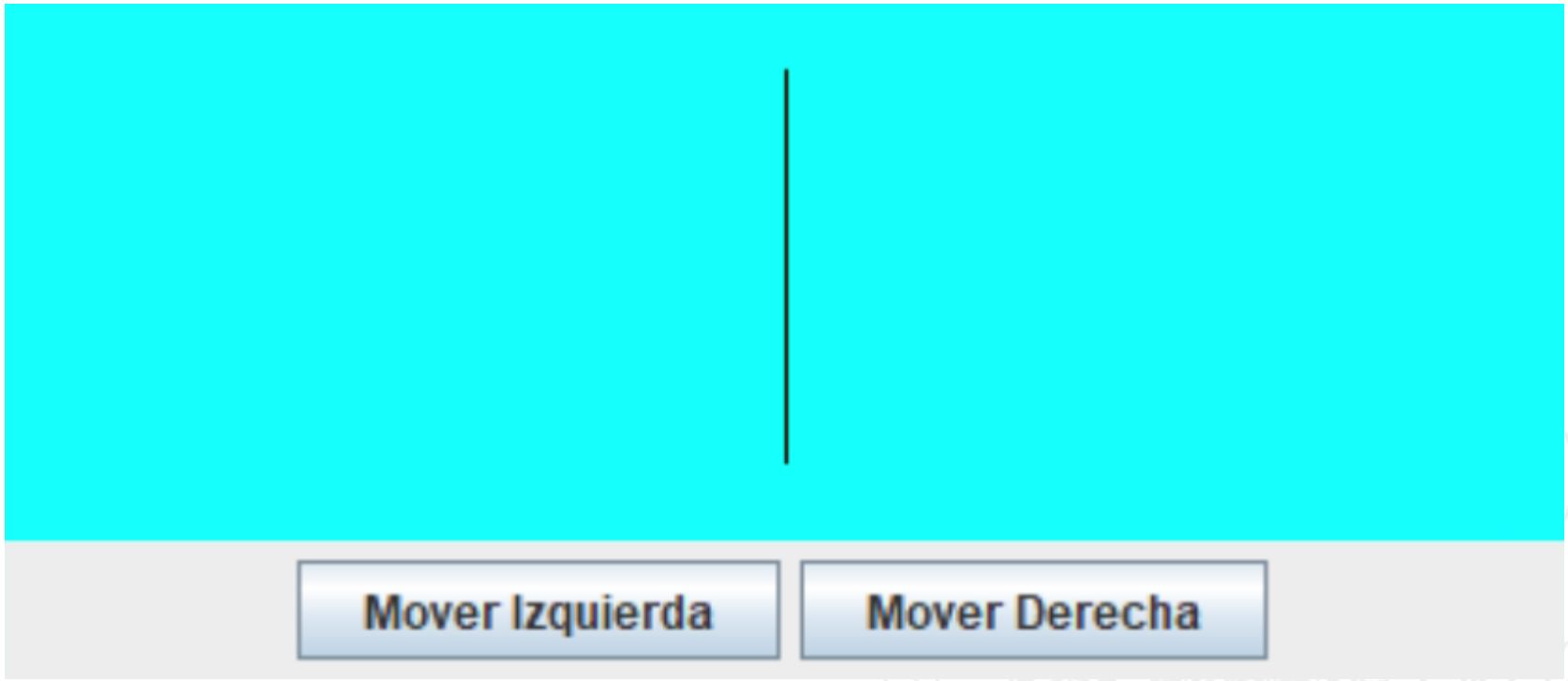
- Ejemplo:

```
JFrame ventana = new JFrame("prueba");
ventana.setLocation(100,100);
ventana.setSize(300,300);
ventana.setVisible(true);
ventana.addMouseListener(new MouseAdapter(){
  public void mouseClicked(MouseEvent e){
    System.out.println("Hola");
  }
});
```

Ejemplo 3. WindowAdapter

```
import javax.swing.*;
import java.awt.event.*;

public class Ej3_WindowAdapter extends JFrame {
    public Ej3_WindowAdapter() {
        setTitle("Ejemplo 3");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel label = new JLabel("Prueba con WindowAdapter");
        add(label);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });

        pack();
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(Ej3_WindowAdapter::new);
    }
}
```

Ejemplo 4. KeyAdapter



La línea se mueve tanto por la acción de los botones del interfaz como por la acción de las teclas de movimiento de cursor del teclado.

Ejemplo 4. KeyAdapter

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Ej4_KeyAdapter extends JFrame {

    private static final int CANVAS_WIDTH = 400, CANVAS_HEIGHT = 140;
    private static final Color LINE_COLOR = Color.BLACK, BG_COLOR = Color.CYAN;
    private int x1 = CANVAS_WIDTH / 2, y1 = CANVAS_HEIGHT / 8;
    private int x2 = x1, y2 = CANVAS_HEIGHT * 7 / 8;
    private DrawCanvas canvas;

    public Ej4_KeyAdapter() {
        setTitle("Mover una Linea");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Creamos el panel de los botones
        JPanel btnPanel = new JPanel(new FlowLayout());
        JButton btnIzq = new JButton("Mover Izquierda");
        btnPanel.add(btnIzq);
        btnIzq.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                x1 -= 10;
                x2 -= 10;
                canvas.repaint();
                requestFocus();
            }
        });
```

Ejemplo 4. KeyAdapter

```
JButton btnDer = new JButton("Mover Derecha");
btnPanel.add(btnDer);
btnDer.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        x1 += 10;
        x2 += 10;
        canvas.repaint();
        requestFocus();
    }
});
// Creamos el canvas
canvas = new DrawCanvas();
canvas.setBackground(BG_COLOR);
canvas.setPreferredSize(new Dimension(CANVAS_WIDTH, CANVAS_HEIGHT));
// Añadimos el canvas y el panel
add(canvas, BorderLayout.CENTER);
add(btnPanel, BorderLayout.SOUTH);
addKeyListener(new KeyAdapter() {
    public void keyPressed(KeyEvent evt) {
        switch (evt.getKeyCode()) {
            case KeyEvent.VK_LEFT:
                x1 -= 10;
                x2 -= 10;
                canvas.repaint();
                break;
            case KeyEvent.VK_RIGHT:
                x1 += 10;
                x2 += 10;
                canvas.repaint();
                break;
        }
    }
});
```

Ejemplo 4. KeyAdapter

```
        pack();
        setVisible(true);
        requestFocus();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(Ej4_KeyAdapter::new);
    }

    class DrawCanvas extends JPanel {
        public void paintComponent(Graphics g) {
            super.paintComponent(g);
            this.setBackground(BG_COLOR);
            g.setColor(LINE_COLOR);
            g.drawLine(x1, y1, x2, y2);
        }
    }
}
```

Ejemplo 4. KeyAdapter

Partiendo del código del Ejemplo 4, realiza las siguientes ampliaciones:

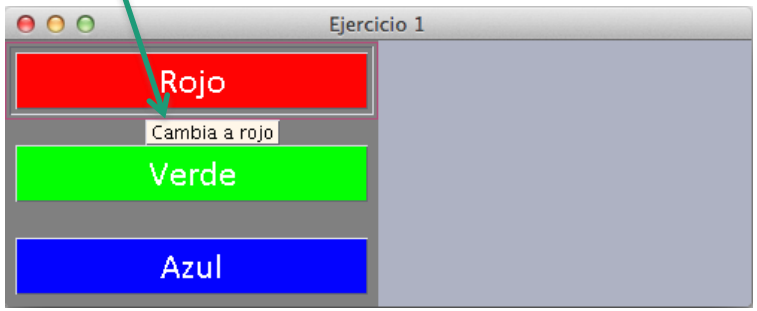
1. **Mover verticalmente.** Añade una segunda línea horizontal en el canvas. Incorpora dos botones "Mover Arriba" y "Mover Abajo" al panel de botones, y haz que las teclas ↑ ↓ del teclado muevan dicha línea horizontal, de forma análoga a como los botones y teclas ← → mueven la línea vertical.
2. **Cambiar el color de la línea.** Añade un botón "Color Aleatorio" que cambie el color de ambas líneas a un color aleatorio cada vez que se pulse.

Práctica 3

1. Utiliza todo lo aprendido hasta ahora para obtener la interfaz de la diapositiva siguiente. Pulsando el botón correspondiente, se debe cambiar el color del panel de la derecha.

2. Integra los botones del ejemplo 4, de tal manera que aparezcan las líneas del ejemplo 4 en el cuadro de la derecha con los respectivos movimientos horizontal y vertical.

Práctica 3

Una vez pulsado, el botón
Quedará desactivado

Los botones deben tener
Texto explicativo